
FUNDACIÓN KINAL

PROYECTO INTEGRADOR

DESARROLLO DE UN SISTEMA BANCARIO

Documentación del proyecto — Guía de trabajo intensivo, 10 días

Datos del proyecto	Información
Institución	Fundación Kinal
Curso / Área	Taller 2
Grado y sección	5to Perito en Programación y Tecnología — Sección: _____
Docente	_____
Equipo / integrantes	_____ _____ _____
Fecha de inicio	___ / ___ / _____
Fecha de entrega	___ / ___ / _____ (Día 10)
Repositorio GitHub	https://github.com/_____/sistema-bancario

Sistema exclusivamente educativo. Todos los datos, cuentas y credenciales son ficticios.

Contenido

1. Descripción del problema y solución
2. Objetivos
3. Requerimientos
4. Historias de usuario
5. Diagramas (casos de uso y flujo)
6. Arquitectura
7. Base de datos y script SQL
8. Endpoints de la API
9. Credenciales ficticias de prueba
10. Instalación y configuración
11. Matriz de pruebas
12. Registro de errores y correcciones
13. Manual de usuario
14. Registro de uso de IA
15. Conclusiones y recomendaciones
16. Plan de trabajo y evidencias

1. Descripción del problema y solución

Problema. Una entidad financiera necesita controlar clientes, cuentas y movimientos de dinero de forma ordenada y segura. Cuando este control es manual o está disperso, aparecen errores de saldo, operaciones sin registro, cuentas sin dueño y falta de control de acceso.

Solución. Un sistema bancario académico en tres capas separadas (frontend, backend y base de datos) que permite gestionar clientes y cuentas, realizar depósitos, retiros y transferencias simuladas, consultar el historial de movimientos y obtener reportes, aplicando validaciones y reglas de negocio. El sistema usa únicamente información ficticia.

Alcance. Incluye los módulos obligatorios: Autenticación, Clientes, Cuentas, Depósitos, Retiros, Transferencias, Movimientos, Consultas, Reportes y Validaciones. No incluye dinero, cuentas, tarjetas ni contraseñas reales.

Tecnologías.

Capa	Tecnología
Frontend	Angular + TypeScript
Backend	Node.js + TypeScript / API REST
Base de datos	MySQL
Pruebas de API	Postman
Control de versiones	Git + GitHub
Diseño	Figma, Canva, Penpot u otra herramienta

2. Objetivos

Objetivo general. Desarrollar un sistema bancario académico funcional que integre frontend, backend y base de datos, aplicando el ciclo de desarrollo de software.

Objetivos específicos.

- Implementar operaciones CRUD de clientes y cuentas.
- Implementar depósitos, retiros y transferencias simuladas con sus validaciones.
- Registrar cada operación con fecha, tipo, monto y cuentas involucradas.
- Implementar autenticación y control de acceso con credenciales ficticias.
- Probar la API con Postman y documentar los resultados como evidencia.
- Usar Git y GitHub con commits frecuentes y descriptivos.
- Documentar el sistema y demostrar individualmente la comprensión del proyecto.

3. Requerimientos

3.1 Requerimientos funcionales

ID	Requerimiento	Módulo
RF01	Iniciar y cerrar sesión validando credenciales ficticias.	Autenticación
RF02	Registrar, consultar, modificar y desactivar clientes.	Clientes
RF03	Crear cuentas asociadas a un cliente.	Cuentas
RF04	Consultar el saldo y actualizar el estado de una cuenta.	Cuentas
RF05	Registrar depósitos y actualizar el saldo.	Depósitos
RF06	Registrar retiros validando el saldo disponible.	Retiros
RF07	Transferir entre cuentas y registrar la operación.	Transferencias
RF08	Consultar el historial de depósitos, retiros y transferencias.	Movimientos
RF09	Buscar clientes y cuentas, y consultar saldo y movimientos.	Consultas
RF10	Mostrar al menos un reporte o resumen de operaciones.	Reportes
RF11	Validar campos obligatorios, datos válidos, saldos y reglas de negocio.	Validaciones

3.2 Requerimientos no funcionales

ID	Requerimiento
RNF01	Separación en tres capas: frontend, backend y base de datos.
RNF02	Uso exclusivo de información ficticia.
RNF03	Las operaciones solo se permiten con sesión iniciada (token).
RNF04	Mensajes de error claros para el usuario.
RNF05	Cada operación queda registrada con fecha, tipo, monto y cuentas involucradas.
RNF06	Las contraseñas se almacenan cifradas (hash), nunca en texto plano.
RNF07	Código versionado en GitHub con commits frecuentes y descriptivos.

3.3 Reglas de negocio

ID	Regla
RN01	Una cuenta debe pertenecer a un cliente.
RN02	No se permite retirar un monto mayor al saldo disponible.
RN03	Los depósitos deben ser mayores que cero.
RN04	Los retiros deben ser mayores que cero.
RN05	Una transferencia debe tener cuenta origen y cuenta destino válidas.
RN06	La cuenta origen debe disponer del saldo suficiente.
RN07	No se debe permitir transferir hacia la misma cuenta.
RN08	Cada operación debe quedar registrada con fecha, tipo, monto y cuentas involucradas.
RN09	Una cuenta inactiva no podrá realizar operaciones.
RN10	El sistema debe utilizar únicamente información ficticia.

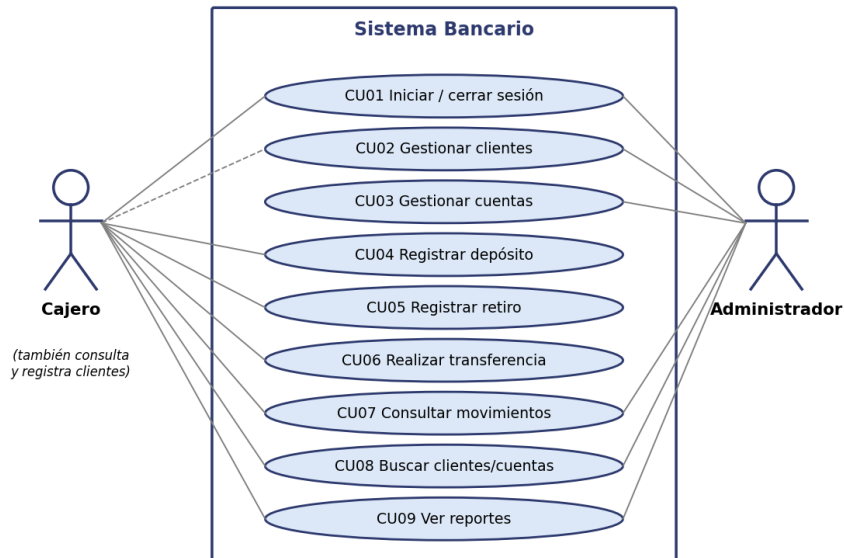
4. Historias de usuario

Formato: *Como [rol], quiero [acción] para [beneficio]*. Cada historia se enlaza con un requerimiento y con la prueba que la valida.

ID	Historia	Criterio de aceptación	RF	Prueba
HU01	Como cajero, quiero iniciar sesión para acceder al sistema.	Con credenciales correctas entra al panel.	RF01	P001
HU02	Como cajero, quiero ver un mensaje de error si mis credenciales son incorrectas para saber que no pude entrar.	Muestra error y no da acceso.	RF01	P002
HU03	Como cajero, quiero cerrar sesión para proteger el sistema.	La sesión termina y redirige al login.	RF01	—
HU04	Como cajero, quiero registrar un cliente para poder abrirle cuentas.	Con datos válidos, el cliente queda almacenado.	RF02	P003
HU05	Como administrador, quiero modificar y desactivar clientes para mantener los datos actualizados.	Los cambios se guardan; el cliente desactivado no se borra.	RF02	—
HU06	Como cajero, quiero crear una cuenta para un cliente existente.	La cuenta queda asociada al cliente.	RF03	P004
HU07	Como cajero, quiero registrar un depósito para actualizar el saldo de una cuenta.	El saldo aumenta; el monto es mayor que cero.	RF05	P005
HU08	Como cajero, quiero registrar un retiro con saldo suficiente para entregar el dinero al cliente.	El saldo disminuye.	RF06	P006
HU09	Como cajero, quiero que se rechace un retiro mayor al saldo para evitar saldos negativos.	La operación se rechaza con mensaje.	RF06	P007
HU10	Como cajero, quiero transferir entre dos cuentas para mover dinero entre clientes.	Se actualizan ambas cuentas y se registra la operación.	RF07	P008
HU11	Como cajero, quiero que no se permita transferir a la misma cuenta para evitar operaciones inválidas.	La operación se rechaza.	RF07	P009
HU12	Como administrador, quiero desactivar una cuenta para impedir que realice operaciones.	La cuenta inactiva rechaza depósitos, retiros y transferencias.	RF04	P010
HU13	Como cajero, quiero buscar clientes y cuentas para atender rápido al cliente.	La búsqueda devuelve coincidencias por nombre, documento o número de cuenta.	RF09	—
HU14	Como cajero, quiero consultar el historial de movimientos de una cuenta para verificar operaciones.	Lista depósitos, retiros y transferencias con fecha, tipo y monto.	RF08	—
HU15	Como administrador, quiero ver un resumen de operaciones para supervisar la actividad.	Muestra cantidad y total por tipo de operación.	RF10	—

5. Diagramas

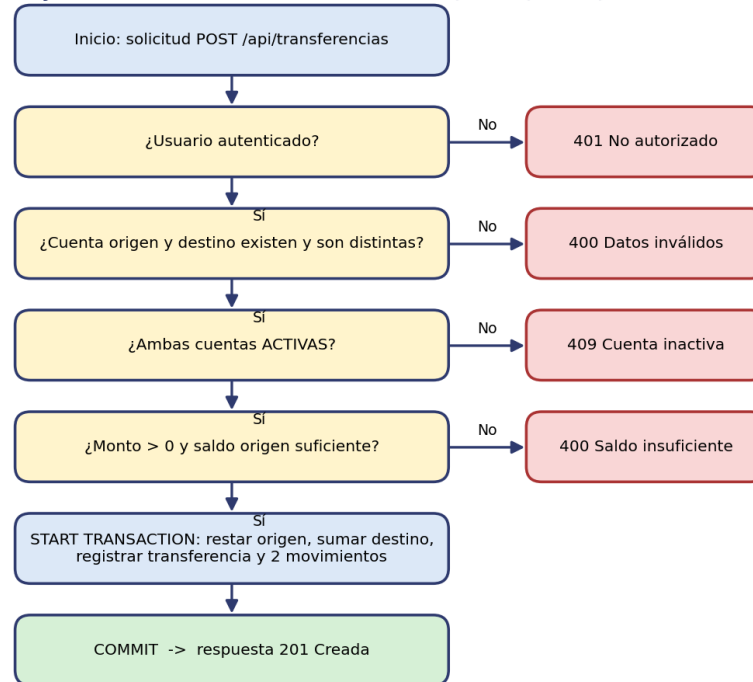
5.1 Casos de uso



Actores: **Cajero** (operaciones diarias) y **Administrador** (gestión de clientes y cuentas, supervisión y reportes).

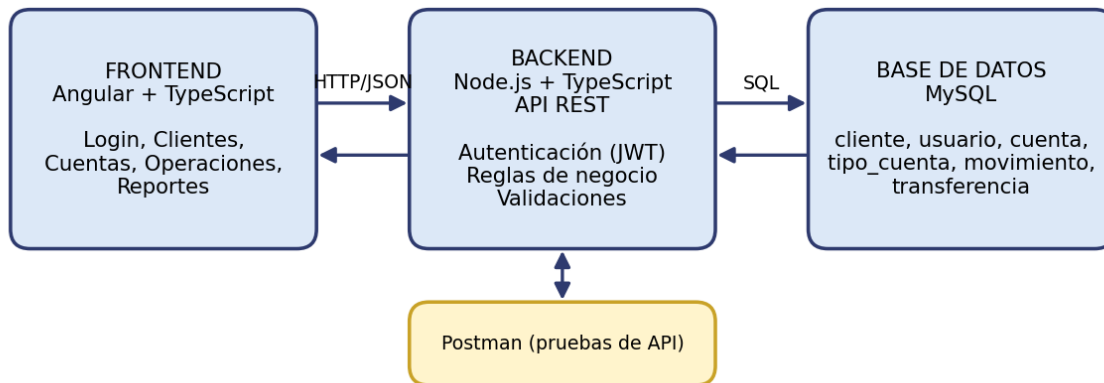
5.2 Flujo de una transferencia

Flujo: transferencia entre cuentas (RN05, RN06, RN07, RN09)



6. Arquitectura

Arquitectura en tres capas - Sistema Bancario Académico

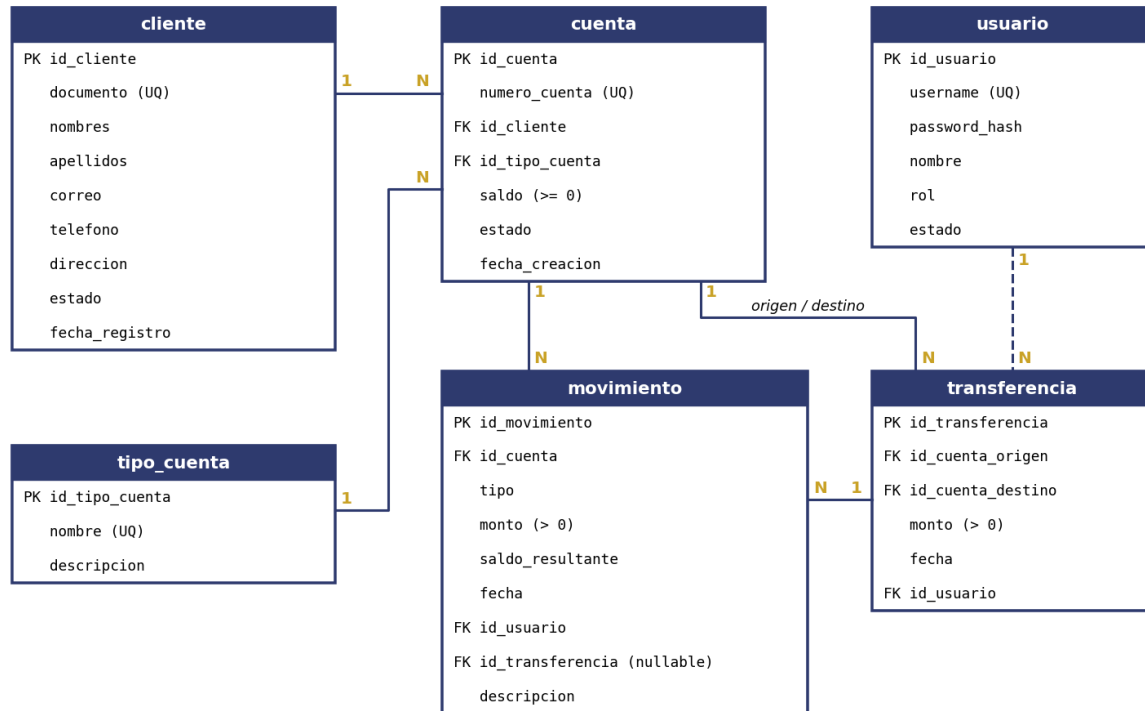


- **Frontend (Angular + TypeScript):** interfaces, formularios, navegación y consumo de la API. Guarda el token de sesión y protege las rutas.
- **Backend (Node.js + TypeScript):** API REST. Valida datos, aplica las reglas de negocio, autentica con token y se conecta a la base de datos.
- **Base de datos (MySQL):** almacena clientes, usuarios, cuentas, movimientos y transferencias con claves primarias, foráneas y restricciones CHECK.
- **Comunicación:** el frontend y el backend intercambian JSON por HTTP; el backend usa SQL para acceder a los datos.
- **Transacciones:** los depósitos, retiros y transferencias se ejecutan dentro de una transacción SQL (todo o nada) para mantener los saldos consistentes.

7. Base de datos y script SQL

7.1 Diagrama entidad-relación

Diagrama Entidad-Relación - banco_kinal



*transferencia.id_cuenta_origen e id_cuenta_destino referencian a cuenta (dos relaciones 1:N).
movimiento.id_usuario y transferencia.id_usuario referencian a usuario (1:N).*

7.2 Modelo lógico

Tabla	Campos (PK en negrita, FK con →)	Restricciones
cliente	id_cliente , documento, nombres, apellidos, correo, telefono, direccion, estado, fecha_registro	documento UNIQUE
usuario	id_usuario , username, password_hash, nombre, rol, estado	username UNIQUE; rol ADMIN/CAJERO
tipo_cuenta	id_tipo_cuenta , nombre, descripcion	nombre UNIQUE
cuenta	id_cuenta , numero_cuenta, id_cliente → cliente, id_tipo_cuenta → tipo_cuenta, saldo, estado, fecha_creacion	numero_cuenta UNIQUE; saldo >= 0; estado ACTIVA/INACTIVA
transferencia	id_transferencia , id_cuenta_origen → cuenta, id_cuenta_destino → cuenta, monto, fecha, id_usuario → usuario	monto > 0; origen <> destino
movimiento	id_movimiento , id_cuenta → cuenta, tipo, monto, saldo_resultante, fecha, id_usuario → usuario, id_transferencia → transferencia (opcional), descripcion	monto > 0; tipo: DEPOSITO, RETIRO, TRANSFERENCIA_ENVIADA, TRANSFERENCIA_RECIBIDA

7.3 Relaciones

- Un cliente tiene muchas cuentas (1:N). Una cuenta pertenece a un solo cliente (RN01).
- Un tipo de cuenta clasifica muchas cuentas (1:N).
- Una cuenta tiene muchos movimientos (1:N). Una transferencia genera dos movimientos: uno enviado y uno recibido.
- Una transferencia referencia dos cuentas: origen y destino. Un usuario registra muchas operaciones (1:N).

7.4 Script SQL

El script completo y funcional (MySQL 8.0.16 o superior) se entrega en el archivo **banco_script.sql**. Crea la base `banco_kinal`, las 6 tablas con sus claves y restricciones, e inserta datos ficticios de prueba.

Fragmento:

```
CREATE TABLE cuenta (  
  id_cuenta      INT AUTO_INCREMENT PRIMARY KEY,  
  numero_cuenta  VARCHAR(12)  NOT NULL UNIQUE,  
  id_cliente     INT           NOT NULL,  
  id_tipo_cuenta INT           NOT NULL,  
  saldo          DECIMAL(12,2) NOT NULL DEFAULT 0.00,  
  estado         ENUM('ACTIVA','INACTIVA') NOT NULL DEFAULT 'ACTIVA',  
  fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente),  
  FOREIGN KEY (id_tipo_cuenta) REFERENCES tipo_cuenta(id_tipo_cuenta),  
  CHECK (saldo >= 0)  
);
```

8. Endpoints de la API

Prefijo base: /api. Formato JSON. Todas las rutas, excepto el login, requieren el encabezado Authorization: Bearer <token>. Las rutas concretas pueden ajustarse si el equipo las cambia; en ese caso actualizar esta tabla.

Recurso	Método y ruta	Descripción	Respuestas
Autenticación	POST /auth/login	Body: username, password. Devuelve token.	200, 400, 401
Cientes	GET /clientes	Lista clientes (filtro opcional ?buscar=).	200
	GET /clientes/:id	Consulta un cliente.	200, 404
	POST /clientes	Registra un cliente.	201, 400, 409
	PUT /clientes/:id	Modifica datos del cliente.	200, 400, 404
	PATCH /clientes/:id/desactivar	Desactivación lógica (no se borra).	200, 404
Cuentas	GET /cuentas	Lista cuentas (filtro opcional ?buscar=).	200
	GET /cuentas/:id	Consulta cuenta y saldo.	200, 404
	POST /cuentas	Crea cuenta: id_cliente, id_tipo_cuenta, saldo_inicial (opcional, se registra como depósito).	201, 400, 404
	PUT /cuentas/:id	Actualiza tipo o estado (ACTIVA/INACTIVA).	200, 400, 404
Depósitos	POST /depositos	Body: id_cuenta, monto, descripcion.	201, 400, 404, 409
Retiros	POST /retiros	Body: id_cuenta, monto. Valida saldo.	201, 400, 404, 409
Transferencias	POST /transferencias	Body: id_cuenta_origen, id_cuenta_destino, monto.	201, 400, 404, 409
Movimientos	GET /movimientos	Lista movimientos. Filtros opcionales: tipo, desde, hasta (AAAA-MM-DD), limite.	200, 400
	GET /movimientos/cuenta/:id	Historial de una cuenta.	200, 404
	POST /movimientos	Registra DEPOSITO o RETIRO (tipo, id_cuenta, monto). Siempre actualiza el saldo. Las transferencias usan /transferencias.	201, 400, 404, 409
Reportes	GET /reportes/resumen	Cantidad y total por tipo de operación y totales del banco. Filtros opcionales: desde, hasta.	200, 400

Códigos usados: 200 OK, 201 Creado, 400 Datos inválidos, 401 No autorizado, 404 No encontrado, 409 Conflicto (cuenta inactiva o saldo insuficiente).

Ejemplo: depósito

```
POST /api/depositos
Authorization: Bearer <token>
```

```
{ "id_cuenta": 1, "monto": 200.00, "descripcion": "Depósito en ventanilla" }
```

```
201 Created
{ "mensaje": "Depósito registrado", "saldo_actual": 1200.00 }
```

9. Credenciales ficticias de prueba

Usuario	Contraseña	Rol
admin	Admin123!	ADMIN
cajero1	Cajero123!	CAJERO

Datos de prueba incluidos en el script SQL:

Cuenta	Cliente	Tipo	Saldo	Estado
1000000000001	Ana López Demo	AHORRO	1000.00	ACTIVA
1000000000002	Carlos Pérez Demo	MONETARIA	2500.00	ACTIVA
1000000000003	María Gómez Demo	AHORRO	500.00	INACTIVA (para la prueba P010)

Son credenciales y datos inventados solo para pruebas académicas. En la base de datos las contraseñas se guardan cifradas (bcrypt).

10. Instalación y configuración

Requisitos: Node.js 18 o superior, MySQL 8.0.16 o superior, Git, Angular CLI, Postman.

Paso 1. Clonar el repositorio

```
git clone <URL_DEL_REPOSITORIO>
cd sistema-bancario
```

Paso 2. Crear la base de datos

```
mysql -u root -p < database/banco_script.sql
```

Paso 3. Configurar y ejecutar el backend

```
cd backend
npm install
cp .env.example .env      # completar los datos de conexión
npm run dev
```

Ejemplo de variables en `.env` (valores de ejemplo):

```
PORT=3000
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=su_clave_local
DB_NAME=banco_kinal
JWT_SECRET=clave_de_ejemplo_cambiar
```

Paso 4. Ejecutar el frontend

```
cd frontend
npm install
ng serve
```

Abrir `http://localhost:4200` e ingresar con las credenciales de la sección 9. Los comandos exactos (por ejemplo, `npm run dev`) deben coincidir con los scripts del `package.json` del proyecto.

Estructura del repositorio

```
sistema-bancario/
|-- frontend/      (Angular + TypeScript)
|-- backend/       (Node.js + TypeScript, API REST)
|-- database/      (banco_script.sql, ERD)
|-- docs/          (documentación, diagramas, evidencias)
|-- postman/       (colección de pruebas)
`-- README.md
```

11. Matriz de pruebas

Las 10 pruebas obligatorias de la guía. Las columnas **Resultado obtenido**, **Estado** y **Evidencia** se completan al ejecutar cada prueba en Postman o en la interfaz (día 7 y día 9).

ID	Prueba	Datos / pasos	Resultado esperado	Resultado obtenido	Estado	Evidencia
P001	Login válido	admin / Admin123!	Permite acceso.		Pendiente	Captura 1
P002	Login inválido	admin / clave_erronea	Muestra mensaje de error.		Pendiente	Captura 2
P003	Registrar cliente	Datos válidos de un cliente nuevo	Cliente almacenado.		Pendiente	Captura 3
P004	Crear cuenta	Cliente existente + tipo AHORRO	Cuenta asociada al cliente.		Pendiente	Captura 4
P005	Depósito válido	Cuenta 100000000001, monto 200	Saldo incrementado.		Pendiente	Captura 5
P006	Retiro con saldo suficiente	Cuenta 100000000001, monto 100	Saldo disminuido.		Pendiente	Captura 6
P007	Retiro superior al saldo	Cuenta 100000000001, monto 999999	Operación rechazada.		Pendiente	Captura 7
P008	Transferencia válida	Origen 100000000001 a destino 100000000002, monto 50	Se actualizan ambas cuentas.		Pendiente	Captura 8
P009	Transferencia a misma cuenta	Origen = destino = 100000000001	Operación rechazada.		Pendiente	Captura 9
P010	Cuenta inactiva	Depósito en cuenta 100000000003	No permite operaciones.		Pendiente	Captura 10

12. Registro de errores y correcciones

Se llena durante el desarrollo: cada error encontrado en las pruebas se documenta con su causa y su corrección.

ID	Fecha	Descripción del error	Módulo	Causa	Corrección aplicada	Responsable	Estado
E01							
E02							
E03							
E04							
E05							
E06							

13. Manual de usuario

Este manual describe el uso del sistema según las funcionalidades planificadas. Ajustar los nombres de botones y pantallas si la interfaz final cambia.

13.1 Iniciar sesión

1. Abrir el sistema en el navegador.
2. Escribir usuario y contraseña (ver credenciales ficticias).
3. Presionar «Ingresar». Si los datos son incorrectos se muestra un mensaje de error.

13.2 Gestionar clientes

1. Entrar al módulo «Clientes».
2. Para registrar: presionar «Nuevo cliente», llenar los campos obligatorios y guardar.
3. Para modificar: elegir el cliente, editar y guardar.
4. Para desactivar: elegir el cliente y presionar «Desactivar». El cliente no se borra.

13.3 Gestionar cuentas

1. Entrar a «Cuentas» y presionar «Nueva cuenta».
2. Elegir el cliente y el tipo de cuenta; indicar el saldo inicial si corresponde.
3. Para cambiar el estado, abrir la cuenta y elegir Activa o Inactiva.

13.4 Registrar un depósito

1. Entrar a «Depósitos», elegir la cuenta.
2. Escribir el monto (mayor que cero) y guardar.
3. El sistema muestra el nuevo saldo.

13.5 Registrar un retiro

1. Entrar a «Retiros», elegir la cuenta.
2. Escribir el monto (mayor que cero y no mayor al saldo) y guardar.
3. Si el saldo no alcanza, la operación se rechaza con un mensaje.

13.6 Realizar una transferencia

1. Entrar a «Transferencias».
2. Elegir la cuenta origen y la cuenta destino (deben ser distintas y estar activas).
3. Escribir el monto y confirmar. Se actualizan ambas cuentas.

13.7 Consultar movimientos y buscar

1. En «Movimientos» elegir una cuenta para ver su historial.
2. Usar el buscador para encontrar clientes o cuentas por nombre, documento o número.

13.8 Ver reportes

1. Entrar a «Reportes» para ver el resumen de operaciones por tipo (cantidad y total).

13.9 Cerrar sesión

1. Presionar «Cerrar sesión» en el menú. El sistema vuelve a la pantalla de inicio de sesión.

14. Registro de uso de IA

La IA se usó como herramienta de apoyo, sin sustituir la comprensión del estudiante. Las columnas **Modificación** y **Comprensión** deben ser completadas por cada integrante con sus propias palabras antes de la entrega.

Campo	Registro 1	Registro 2	Registro 3
Herramienta	Claude (Anthropic)	Claude (Anthropic)	Claude (Anthropic)
Prompt	Entregar el Día 1 de la guía (documento inicial y repositorio).	Entregar el Día 2 (requerimientos e historias de usuario).	Generar los documentos de la guía: PDF de documentación, README y script SQL.
Resultado	Documento inicial, objetivos, roles y comandos Git para crear el repositorio.	Requerimientos, reglas de negocio y 12 historias de usuario enlazadas a pruebas.	Documentación completa, diagramas, script SQL con datos ficticios y README.
Aplicación	Se usó como base del documento inicial y de la estructura del repositorio.	Se usó como base de requerimientos e historias.	Se usó como base de la documentación final y del modelo de datos.
Modificación	(completar: qué cambió el estudiante)	(completar)	(completar)
Comprensión	(completar: explicación propia)	(completar)	(completar)

15. Conclusiones y recomendaciones

Borrador para completar y ajustar al finalizar el proyecto con los resultados reales.

Conclusiones

- El proyecto permitió integrar frontend, backend y base de datos en una solución funcional, aplicando el ciclo de desarrollo de software.
- Las validaciones y reglas de negocio garantizan la consistencia de los saldos y el registro de cada operación.
- Las pruebas obligatorias (P001 a P010) permitieron comprobar el comportamiento esperado del sistema.
- El uso de Git y GitHub dejó evidencia del aporte de cada integrante durante los 10 días.

Recomendaciones

- Agregar roles y permisos más detallados, y recuperación de contraseña.
- Automatizar las pruebas de la API para no depender solo de Postman.
- Incluir paginación y filtros por fecha en los movimientos.
- Mantener la documentación actualizada con cada cambio del código.

16. Plan de trabajo y evidencias

Día	Trabajo obligatorio	Evidencia / entrega	Responsable	Estado
1	Definir problemática, alcance, integrantes, roles y objetivos.	Documento inicial + repositorio creado.		
2	Requerimientos e historias de usuario.	Mínimo 10 historias + requerimientos.		
3	Diseño de arquitectura, casos de uso y flujos.	Diagramas + arquitectura.		
4	Diseño de BD y creación de tablas.	ERD + script SQL.		
5	Backend: estructura, conexión BD y CRUD inicial.	API funcional + commits.		
6	Backend: autenticación y operaciones bancarias.	Depósitos, retiros y validaciones.		
7	Transferencias, movimientos y pruebas de API.	Postman + evidencias.		
8	Frontend: login, clientes, cuentas y formularios.	Interfaces conectadas a API.		
9	Integración completa, pruebas, correcciones y documentación.	Sistema integrado + matriz de pruebas.		
10	Mantenimiento final, README, manual, presentación y defensa.	Entrega final + exposición.		

Evidencia diaria: commits frecuentes y descriptivos, tareas con responsable, capturas de pruebas, documentación actualizada y aportes demostrables de cada integrante.